NEURAL RECOGNITION OF PROPERTIES IN NETWORK VISUALIZATIONS

By
BRIANNA NICOLE YUKI

A Thesis Submitted to The W.A. Franke Honors College

In Partial Fulfillment of the bachelor's degree
With Honors in

Computer Science

THE UNIVERSITY OF ARIZONA

M A Y 2 0 2 6

Approved by:
Reyan Ahmed
Department of Computer Science

Neural Recognition of Properties in Network Visualizations

Brianna Nicole Yuki, Reyan Ahmed

Network visualization plays a significant role in understanding the topology of a network. Different visualizations exhibit distinct properties depending on the intended application. We have developed neural-network-based models to recognize several fundamental properties of network visualizations. We investigate the capabilities of simple neural networks to identify these properties. We also develop models to determine the layout size based on the density distribution. The models are trained and tested on approximately one million images. We also analyze the key factors that improve model accuracy. Based on this analysis, we develop a novel framework that can determine different properties of network layouts of different sizes. Finally, we interpret the predictions of the models using feature attribution methods.

1 Introduction

Networks appear in many different fields, for example, roads, power grids, and social networks. These are commonly visualized using graphs, which can help to understand these networks. However, large networks can be complex, containing a lot of nodes and edges, which could make it difficult for a human to determine if a network visualization contains certain properties. In some situations, you may only have access to an image of a graph and not the underlying structural data, which may require you to manually analyze the graph, which can be error-prone and might take a lot of time. Neural networks have been extremely successful in computer vision [16, 18, 23]. By using neural networks, classifications of the images of graphs could be done more quickly. There has been previous work using neural networks for graph classification, however, it has mainly been done using Graph Neural Networks [6]. GNNs typically take adjacency lists or matrices as input, whereas we are using images of networks as input, so the model doesn't know the exact structure of the graph and has to rely on pixel values.

In recent years, an extensive amount of work has been conducted to apply different learning algorithms to network visualization. Stochastic gradient descent, one of the most popular algorithms for learning, has been successfully applied to visualize network layouts with different aesthetic properties [1, 2, 8, 26]. The recent success of large language models may also be extended to this domain. Researchers are adopting advanced techniques and improving performance using various quality metrics. Machine learning–based packages are frequently updated to enhance algorithms, while previous implementations are becoming obsolete or difficult to replicate.

Overall, these rapid changes create obstacles to reproducing previous experiments. It is becoming increasingly difficult to re-implement existing algorithms for reproducibility purposes or to compare them with other approaches, as they often depend on a wide range of parameters. However, the images and evaluation results of previous visualization methods can often be obtained from research articles and related code repositories. If we can directly measure quality metrics from the images, this would bypass the need to re-implement the algorithms.

In this work, we study how well a neural network can determine whether a graph exhibits a specific property given only its image. Two of the properties we consider are relatively simple: connectivity and tree recognition. Given a graph $G = (V, E)$, these properties can be determined using depth-first search in $O(|V| + |E|)$ time [7]. However, the main challenge here is that we are not given G; instead, we only have an image of a graph layout. Graph connectivity is related to the community detection problem, which has many practical applications [5, 19]. We also study the network's ability to recognize planarity. Planar graphs are well studied, and there are many celebrated results concerning them [13]. We selected these three tasks to evaluate the effectiveness of our framework; however, other properties could also be incorporated.

The emergence of recent AI tools and their popularity in day-to-day problem solving have raised questions

about their trustworthiness. Recent work has proposed different visual explanations for such AI tools [10]. Overall, it is important to understand why an AI tool or model makes a particular decision. Decision trees have been used to explain model predictions. Learning algorithms select different features from the input, and predictions are made based on these selected features. However, models can mistakenly correlate unrelated features with predictions. For example, models may become biased by background or other dataset artifacts [27]. Hence, it is important to examine which features models rely on. In fact, several methods have been proposed for model explainability [4, 25], and we use these methods in our work.

Modern vision architectures exhibit improved flexibility toward varying resolutions. However, when network size (e.g., the number of nodes) changes, it is not only the resolution that is affected, but the recognition task itself also changes significantly. For example, in a dataset of 20-node graphs, finding a connected subgraph of size 20 without cycles implies that the graph is a tree. If the model learns this pattern and later detects a subtree of twenty nodes within a larger network, it may incorrectly classify the entire network as a tree. To address this issue, we first train a model to estimate the size of the network. Previous work has used convolutional neural networks to count the number of objects by learning density maps [17]. We adopt a similar approach and decompose the property recognition task into two main steps. The first step estimates the size of the network, and the second step applies a model to recognize properties conditioned on that size (see Figure 1).

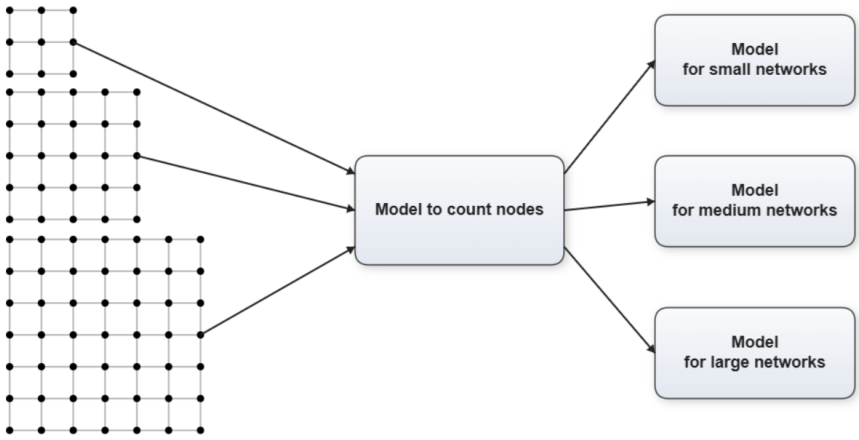

Figure 1: Our framework takes a network layout and counts the number of nodes in the first step. Then, it uses the appropriate model to recognize network properties.

In summary, we use simple neural networks to address several fundamental classification tasks. Given a network layout, we aim to determine whether the image represents a connected network. We also assess whether the image corresponds to a tree and whether it represents a planar graph. We refer to these three tasks as Connectivity, Tree Recognition, and Planarity Recognition, respectively. We propose a framework that can recognize these properties across network images of varying sizes.

2 Our Method

In this section, we describe the models and regularization techniques considered, along with the method used to determine network size and interpret model outputs.

2.1 Neural Network Models

We consider several models that use different types of layers, activation functions, and loss functions. We consider fully connected layers and convolutional layers, as well as sigmoid and ReLU activations. We also consider mean squared error (MSE), binary cross-entropy, and cross-entropy with softmax losses. We provide the detailed architecture of our models below.

Simple NN: This is a simple neural network (simple NN) model that uses a fully connected neural network for binary classification. It takes flattened 28×28 (or 56×56 for larger) grayscale images as input and passes them through two hidden layers. The first hidden layer maps 784 (or 3136) inputs to 128 neurons, and the second hidden layer maps 128 features to 64 neurons. The final layer maps 64 features to a single output neuron. This model uses sigmoid activations and MSE loss. Note that this model is very similar to one proposed for the MNIST dataset [20], except that the output layer has been adapted for our task.

BinaryModel: This model defines a simple fully connected (dense) neural network for binary classification. We use PyTorch [22] for this and the following two models. It takes 28×28 (or 56×56 for larger) input images (such as grayscale digit images), flattens them into a 784 (or 3136)-dimensional vector, and passes them through three linear (dense) layers. The first layer maps the input vector to 128 hidden units, followed by a ReLU activation; the second reduces 128 features to 64, again with ReLU; and the final layer maps 64 features to a single output neuron. Because the model returns raw logits (no sigmoid in the forward pas), it is paired with binary cross-entropy with logits loss, which internally applies a numerically stable sigmoid activation before computing binary cross-entropy. Overall, this is a straightforward multilayer perceptron (MLP) suitable for binary classification on flattened image inputs.

BinaryCEModel: This model defines a fully connected neural network that treats binary classification as a two-class problem rather than using a single-logit output. The input images of size 28×28 (or 56×56 for larger layouts) are flattened into 784 (or 3136)-dimensional vectors and passed through two hidden layers of sizes 128 and 64, each followed by ReLU activation. The final layer outputs two logits (instead of one), representing the scores for each class. The model is trained with cross-entropy loss. The outputs are raw logits, and no softmax is applied in the forward pass; the cross-entropy loss internally applies log-softmax in a numerically stable way. Conceptually, this setup treats binary classification as a special case of multi-class classification with two categories, learning to assign higher probability to the correct class via a softmax-based decision rule.

Convolutional model: This model defines a simple convolutional neural network (CNN) for two-class image classification. It takes grayscale 28×28 (or 56×56 for larger) images as input and first applies two convolutional layers: the first maps 1 channel to 16 feature maps, and the second maps 16 to 32, both using 3×3 kernels with padding to preserve spatial dimensions. Each convolution is followed by a ReLU activation and a 2×2 max-pooling operation, reducing the spatial resolution by half at each stage while increasing feature depth. The resulting feature maps are flattened and passed through a fully connected layer with 128 units, followed by a final linear layer producing two logits corresponding to the two classes. As in the previous model, cross-entropy loss is used. The model combines convolutional feature extraction with dense classification layers for end-to-end learning.

2.2 Regularization Techniques

We now describe the regularization techniques used.

L_2 **regularization:** In one of our configuration we apply L_2 regularization, commonly referred to as weight decay. L_2 regularization works by adding a penalty proportional to the square of the magnitude of the model's weights to the loss function. This discourages the network from assigning excessively large values to parameters, effectively constraining the model's complexity. As a result, it helps reduce overfitting by encouraging smoother, more generalizable solutions rather than ones that memorize the training data. In practice, L_2 regularization improves stability during training and often leads to better performance on unseen data. We use L_2 regularization with strength 10^{-4}.

Dropout regularization: In one of our configurations, we employ dropout regularization with a dropout probability of 0.5 applied after the first fully connected layer. Dropout works by randomly deactivating (setting to zero) 50% of the neurons during each training iteration, which prevents the network from becoming overly dependent on specific features or neuron activations. By forcing the model to rely on different subsets of neurons in each forward pass, dropout reduces co-adaptation between neurons and encourages the learning of more robust, distributed feature representations. This helps mitigate overfitting and improves the model's ability to generalize to unseen data. Importantly, dropout is only active during training and is automatically disabled during evaluation, ensuring that all learned features are used when making predictions.

Batch normalization: In one of our configuration, we use batch normalization, a technique that normalizes the activations of a neural network layer across each mini-batch so that they have approximately

zero mean and unit variance. By stabilizing the distribution of intermediate activations during training, it reduces internal covariate shift and allows the network to train faster and with higher learning rates. In addition to improving optimization, Batch Normalization acts as an implicit regularizer because the mini-batch statistics introduce slight noise into the training process, which can reduce overfitting and improve generalization. It also includes learnable scale and shift parameters, enabling the model to preserve representational flexibility while benefiting from normalized inputs. We use the default parameters of PyTorch [22].

2.3 Counting using Density Maps

The counting task refers to the problem of determining the number of objects in a given image. Many previous works have been done using multi-scale architectures which is where a neural processes data at different scales such as resolution or granularity. Zhang et al. [29] present a multi-column neural network (MCNN) for this counting task. Multiple independent columns process the same image and the columns' outputs are combined into a single output. Each column has a different filter size to account for objects of different sizes.

Sam et al. [3] also introduce a multi-column neural network, however they select a single column for the output unlike Zhang et al. [29] whose model combines the outputs of multiple columns. Each column is a CNN with different filter sizes. A patch from a grid of an image gets inputted into a classification model which determines the best CNN model for that patch. The patch gets inputted into the selected CNN model which outputs the count.

Both these models perform well, however they both use MCNNs and which have some drawbacks. Because of the complex structure of MCNNs, they can take a long time to train and the structure can be inefficient, especially for networks with more layers.

Convolutional neural networks [17] have been successfully deployed for this task while overcoming the issues mentioned above. One of the key challenges is that object density can vary significantly across different regions of an image. To address this, a convolutional neural network is trained to learn a density function over the image. Once the density function is learned, it can be integrated over the entire image to obtain the total object count. We use this method to count the number of nodes in a given image as the first step of our framework. This count then helps us decide which neural network to use for predicting graph properties in subsequent steps.

2.4 Model Interpretability

Feature attribution is a way of interpreting models by scoring each feature depending on how it contributes to a model's prediction. In our case, it scores each pixel, allowing us to understand which pixels caused a model to classify an image a certain way.

Sundararajan et al. [25] propose 2 characteristics that a feature attribution method should have: sensitivity and implementation invariance. Sensitivity means that a feature should be nonzero if the input and baseline differ by only that one feature but result in different predictions. Implementation invariance means that networks that are functionally equivalent should have the same attributions. Most methods in previous works do not have both of these characteristics.

Zeiler et al. [28] propose a deconvolutional neural network model. This is similar to a convolutional neural network, however pooling and filtering are done in reverse which expands spatial information rather than compressing it. In this model, if the ReLU node input is negative, then it won't get backpropogated. Because of this, this method does not satisfy sensitivity. Shrikumar et al. [24] introduce DeepLIFT which is a model that uses the difference between the activation of each neuron and the baseline to determine contribution scores. This method does satisfy sensitivity, however does not satisfy implementation invariance.

To interpret our models, we use the integrated gradients technique [25] that satisfies both sensitivity implementation invariance that are important for effective feature attribution. Integrated gradients is a feature attribution method for interpreting neural network predictions by quantifying the contribution of each input feature to a specific output. The method computes attributions by integrating the gradients of the model's output with respect to the input along a straight-line path from a baseline input (typically a zero or neutral reference) to the actual input. This approach captures how the prediction changes as the input transitions from the baseline to its observed value, providing a more stable and theoretically grounded

alternative to single-point gradient methods. Integrated gradients satisfy important axioms such as sensitivity and implementation invariance, ensuring that irrelevant features receive zero attribution and that functionally equivalent models yield consistent explanations. As a result, it is widely used for interpreting deep learning models across domains such as computer vision, natural language processing, and scientific data analysis.

3 Experimental Result

We use the Erdős–R'enyi graph generation model [9] for our experiments. To generate the layouts, we use the classical spring layout algorithm [11]. Since the spring layout distributes nodes according to a particular strategy, to assess how much node positions impact the model's decisions, we also use a random layout. Additionally, we consider two setups: in one setup we keep the number of nodes fixed, and in another setup we vary the number of nodes. We evaluate several models with different architectures and apply different types of regularization. We describe the details of dataset generation, models, and regularization techniques below.

3.1 Dataset Generation

For the Connectivity task, we use the well-known Erdős–R'enyi model [9]. In this model, the number of nodes is first selected, and then an edge is added independently between each pair of nodes with a given probability. For the fixed-node experiment, we set the number of nodes to 10. For positive-class data points (connected graphs), we use an edge probability of 60%. Note that the Erdős–R'enyi model does not guarantee the generation of a connected graph with a 60% edge probability; therefore, we repeatedly generate instances until a connected graph is obtained for each such data point. To generate negative-class data points (disconnected graphs), we use an edge probability of 40%. Again, we ensure that each data point is disconnected by generating multiple instances if necessary.

We use two different algorithms to generate graph layouts. One method is a force-directed algorithm [15]. Among the various force-directed algorithms, we use the NetworkX [12] implementation of the classical Fruchterman–Reingold algorithm [11]. This algorithm treats nodes as if they repel each other like charged particles, while edges act like springs that pull connected nodes closer together. Through an iterative simulation process, it adjusts node positions until the system stabilizes, producing a visually balanced and readable arrangement. This method naturally reveals clusters and relationships in the network, making it especially useful for visualizing graphs that do not have predefined spatial coordinates.

To evaluate the impact of layout choice across different tasks, we also consider random layout generation. We use the NetworkX [12] implementation, which assigns positions to graph nodes completely at random. It places each node independently within a unit square using uniformly distributed coordinates, without considering edges or network structure. Because it does not attempt to optimize spacing or reveal connectivity patterns, the resulting visualization typically appears scattered and unstructured. This layout is mainly useful as a simple baseline, for testing purposes, or as an initial configuration for more advanced layout algorithms that iteratively refine node positions.

In total, we generate 70,000 data points, with an equal number from each class. The training and testing datasets contain 60,000 and 10,000 data points, respectively. To evaluate the effect of node size, we also generate data points with node sizes randomly sampled from $[5, 12]$. For varying node sizes, we generate two additional datasets of the same size for the two layout methods.

For the tree recognition task, we use the NetworkX [12] implementation of the RANRUT algorithm [21]. This algorithm generates a uniformly random unlabeled rooted tree with n nodes, where n is a positive integer. The method is recursive: it considers all partitions of $n-1$ into subtree sizes, assigns each partition a probability proportional to the number of trees consistent with that structure, and samples one accordingly. Each subtree in the selected partition is then generated recursively. This weighting ensures that each unlabeled rooted tree of size n is produced with equal probability. Half of the dataset for this task consists of trees, while the remaining half consists of non-trees. To generate graphs that are not trees, we randomly select a probability from $[0.05, 0.4]$ and use it to generate a graph with n nodes using the Erdős–R'enyi model. If the generated graph is a tree, we iterate over node pairs to find a pair that does not have an edge and add an edge between them.

For the planarity task, we again generate random graphs using the Erdős–R'enyi model. We generate an equal number of planar and non-planar graphs. Since the number of edges in planar graphs is relatively small, we use a low edge probability (0.15). However, there is still a chance of generating a non-planar graph; therefore, we continue generating graphs until we obtain a planar graph for each data point. For generating non-planar graphs, we randomly select a probability from $[0.1, 0.5]$ and repeatedly generate graphs until a non-planar graph is obtained.

For each task, we consider four different datasets of 70,000 data points each. Each dataset uses one layout method: either the force-directed method or the random method; and either a fixed node size or varying node sizes, as described above.

We further extend our dataset to include larger networks. Specifically, we consider networks of sizes 20, 50, and 100. For these larger networks we lowered the edge probabilities so that the sparsity of the networks are more consistent across different network sizes. To train the convolutional network for the density map learning task, we generate a dataset of 10,000 layouts. For each layout, the number of nodes is randomly selected between 10 and 100. Each graph is generated using the Erdős–Rényi model, with the edge probability randomly chosen in the range 0.1–0.4. We use an 80%–20% train–test split.

3.2 Data Transformation

We use different types of data transformations to make the models more robust. For both training and testing, we resize the images to 28×28 so that all images have the same dimensions. For the training dataset, we randomly rotate the images within a range of $\pm 10^\circ$ and randomly scale them from 90% to 110%. For the test dataset, we randomly translate the images horizontally and vertically up to 10% of the image width and height, respectively. These transformations can create empty corners in the images, which we fill with white color. Both the training and testing images are normalized so that the pixel values lie in the range ± 1.

3.3 Training Method

We randomly shuffle the training dataset before training the models. In each epoch we pass a batch of 32 samples to the model instead of just one sample to improve the training time. We use the adaptive moment estimation optimizer (Adam) with the default learning rate 0.001 [14]. We use PyTorch [22] for implenting our models.

3.4 Result

We trained different models using graphs of sizes 10-100. We first trained 8 types of models using graphs containing 10 nodes and graphs containing a random number of nodes: simple NN, BinaryModel, BinaryCE-Model, CNN, CNN with augmented data, CNN with weight decay, CNN with dropout, and CNN with batch normalization. Model performace was measured using test accuracy and test loss.

Input Image				
Model Output	0.9995	0.0018	0.6711	0.3958
Model Prediction	1	0	1	0
Correct Label	1	0	0	1

Table 1: This shows some sample input images and outputs for the simple NN model for the connectivity task using 10 node, spring layout graphs. The first two images were correctly classified and the last two images were incorrectly classified. Notice how the model output for the last two images are closer to 0.5 meaning the model is less confident.

Input Image				
Model Output	0.9682	1.0000	0.8280	0.5633
Model Prediction	1	0	1	0
Correct Label	1	0	0	1

Table 2: This shows some sample input images and outputs for the simple NN model for the tree recognition task using 5-12 node, random layout graphs. The first two images were correctly classified and the last two images were incorrectly classified.

Input Image				
Model Output	0.9693	1.0000	0.7556	0.8594
Model Prediction	1	0	1	0
Correct Label	1	0	0	1

Table 3: This shows some sample input images and outputs for the CNN with dropout model for the planarity recognition task using 10 node, random layout graphs. The first two images were correctly classified and the last two images were incorrectly classified.

We vary the number of nodes from 5-12 to show the performance of the models for a dataset with varying size. For the models that used a range of graph sizes, they performed worse than the corresponding model using a fixed node size. In general, the models for random layout performed worse than the corresponding spring layout model. All the CNN models performed very similarly with the CNN with batch normalization model performing the best the most. We show the test accuracy and loss in Tables 4 and 5 respectively.

Model	**Is Connected**				**Is Tree**				**Is Planar**			
	SL 10	SL 5-12	RL 10	RL 5-12	SL 10	SL 5-12	RL 10	RL 5-12	SL 10	SL 7-12	RL 10	RL 7-12
Simple NN (without pytorch)	0.9962	0.9824	0.9187	0.6840	0.9419	0.8636	0.8378	0.6835	0.9063	0.8969	0.8844	0.8099
BinaryModel	0.9995	0.9966	0.9441	0.7577	0.9537	0.8960	0.8701	0.7546	0.9424	0.9278	0.9006	0.8275
BinaryCEModel	0.9996	0.9961	0.9416	0.7424	0.9595	0.8889	0.8764	0.7603	0.9395	0.9274	0.9024	0.8242
CNN	0.9998	0.9993	0.9547	0.9232	0.9878	0.9863	0.9143	0.8807	0.9460	0.9618	0.9150	0.9267
CNN with Augmented Data	1.0000	0.9993	0.9406	0.9180	0.9844	0.9859	0.9180	0.8788	0.9488	0.9663	0.9130	0.9280
CNN with Weight Decay	0.9999	0.9992	0.9576	0.9205	0.9824	0.9865	0.9156	0.8755	0.9523	0.9640	0.9159	0.9262
CNN with Dropout	0.9998	0.9995	0.9518	0.9270	0.9860	0.9852	0.9193	0.8833	0.9505	0.9649	0.9159	0.9261
CNN with Batch Normalization	1.0000	0.9992	0.9602	0.9237	0.9859	0.9902	0.9163	0.8677	0.9546	0.9642	0.9087	0.9262

Table 4: Test accuracy across graph types and layouts. Input images are 28×28. SL = Spring Layout, RL = Random Layout.

Model	Loss	Is Connected				Is Tree				Is Planar			
		SL 10	SL 5-12	RL 10	RL 5-12	SL 10	SL 5-12	RL 10	RL 5-12	SL 10	SL 7-12	RL 10	RL 7-12
Simple NN (without pytorch)	MSE	0.0032	0.0138	0.0613	0.1959	0.0470	0.0994	0.1191	0.1981	0.0717	0.0780	0.0843	0.1288
BinaryModel	BCE with logits	0.0010	0.0111	0.1526	0.4473	0.1330	0.2498	0.3124	0.4759	0.1386	0.1744	0.2283	0.3527
BinaryCEModel	Cross Entropy	0.0008	0.0147	0.1501	0.4577	0.1184	0.2592	0.3008	0.4721	0.1408	0.1750	0.2322	0.3659
CNN	Cross Entropy	0.0004	0.0014	0.1192	0.1929	0.0365	0.0375	0.2297	0.2926	0.1269	0.0954	0.1999	0.1771
CNN with Augmented Data	Cross Entropy	0.0000	0.0016	0.1479	0.1967	0.0434	0.0444	0.2195	0.2979	0.1151	0.0855	0.2027	0.1747
CNN with Weight Decay	Cross Entropy	0.0002	0.0023	0.1135	0.1938	0.0498	0.0411	0.2302	0.3062	0.1122	0.0959	0.1948	0.1732
CNN with Dropout	Cross Entropy	0.0006	0.0013	0.1344	0.1859	0.0429	0.0427	0.2215	0.2869	0.1146	0.0893	0.1987	0.1852
CNN with Batch Normalization	Cross Entropy	0.0002	0.0020	0.1089	0.1877	0.0437	0.0286	0.2233	0.3149	0.1090	0.0876	0.2035	0.1731

Table 5: Test loss values across graph types, layouts, and node ranges. Input images are 28×28. SL = Spring Layout, RL = Random Layout.

We additionally trained 4 types of models each on graphs containing 20, 50, and 100 nodes: simple NN, BinaryModel, BinaryCEModel, and CNN with batch normalization. These are the same models as before except only using the best performing CNN model. The reason we trained new models is because the models that were trained for 10 node graphs didn't perform well on the 20 node graphs, and similarly, the models trained on the 20 node graphs didn't perform well 50 node graphs as shown in Tables 6 and 7. For the models trained on 20 and 50 nodes, the input images had a size of 28×28. However, the models trained on 100 node images did not perform well when using 28×28 images, so they were instead trained on 56×56 images.

Model	Is Connected		Is Tree		Is Planar	
	SL	RL	SL	RL	SL	RL
Simple NN (without pytorch)	0.9743	0.5000	0.4941	0.4926	0.5445	0.5006
BinaryModel	0.9688	0.5000	0.4491	0.4957	0.5213	0.5001
BinaryCEModel	0.9662	0.5000	0.4290	0.4961	0.5268	0.5003
CNN with Batch Normalization	0.9991	0.5000	0.4997	0.4993	0.5022	0.5028

Table 6: The accuracy for 20 node graphs when using models trained on 10 node graphs. IC = Is Connected, IT = Is Tree, IP = Is Planar, SL = Spring Layout, RL = Random Layout.

Model	Is Connected		Is Tree		Is Planar	
	SL	RL	SL	RL	SL	RL
Simple NN (without pytorch)	0.5746	0.5552	0.4778	0.4977	0.6021	0.5019
BinaryModel	0.5609	0.5693	0.4185	0.4995	0.5916	0.5004
BinaryCEModel	0.5582	0.5631	0.4191	0.4994	0.5895	0.5004
CNN with Batch Normalization	0.8369	0.5997	0.5097	0.5000	0.6638	0.5012

Table 7: The accuracy for 50 node graphs when using models trained on 20 node graphs. IC = Is Connected, IT = Is Tree, IP = Is Planar, SL = Spring Layout, RL = Random Layout.

Input Image				
Model Output	0.8277	0.0000	0.8329	0.4189
Model Prediction	1	0	1	0
Correct Label	1	0	0	1

Table 8: This shows some sample input images and outputs for the BinaryModel for the tree recognition task using 20 node, random layout graphs. The first two images were correctly classified and the last two images were incorrectly classified.

Input Image				
Model Output	0.9407	0.9999	0.8775	0.6067
Model Prediction	1	0	1	0
Correct Label	1	0	0	1

Table 9: This shows some sample input images and outputs for the CNN with batch normalization model for the planarity recognition task using 50 node, random layout graphs. The first two images were correctly classified and the last two images were incorrectly classified.

Input Image				
Model Output	1.0000	1.0000	0.5240	0.8136
Model Prediction	1	0	1	0
Correct Label	1	0	0	1

Table 10: This shows some sample input images and outputs for the BinaryCEModel for the tree recognition task using 100 node, spring layout graphs. The first two images were correctly classified and the last two images were incorrectly classified.

For graphs containing 20 nodes, the CNN with batch normalization model performed the best for all tasks. For all tasks, all models trained on the random layout graphs performed worse except for the simple NN model for the planarity recognition task. We show the test accuracy and loss in Tables 11 and 12 respectively.

Model	**Is Connected**		**Is Tree**		**Is Planar**	
	SL 20	RL 20	SL 20	RL 20	SL 20	RL 20
Simple NN (without pytorch)	0.9999	0.9770	0.9546	0.8478	0.8773	0.8817
BinaryModel	0.9999	0.9894	0.9900	0.8949	0.9074	0.8964
BinaryCEModel	0.9999	0.9897	0.9851	0.8969	0.9047	0.8979
CNN with Batch Normalization	0.9999	0.9937	0.9910	0.9380	0.9132	0.9027

Table 11: Test accuracy for 20 node graphs across different graph types and layouts. Input images are 28×28. IC = Is Connected, IT = Is Tree, IP = Is Planar, SL = Spring Layout, RL = Random Layout.

Model	**Loss**	**Is Connected**		**Is Tree**		**Is Planar**	
		SL 20	RL 20	SL 20	RL 20	SL 20	RL 20
Simple NN (without pytorch)	MSE	0.0003	0.0186	0.0371	0.1124	0.0929	0.0901
BinaryModel	BCE with logits	0.0001	0.0321	0.0350	0.2668	0.2368	0.2590
BinaryCEModel	Cross Entropy	0.0001	0.0329	0.0483	0.2612	0.2365	0.2552
CNN with Batch Normalization	Cross Entropy	0.0006	0.0178	0.0324	0.1839	0.2167	0.2472

Table 12: Test loss values for 20 node graphs across different graph types and layouts. Input images are 28×28. SL = Spring Layout, RL = Random Layout.

For graphs containing 50 nodes, the CNN with batch normalization model also performed the best for all tasks. The easiest task for the models was the connectivity task with spring layout graphs while the hardest task was the tree recognition task with random layout graphs. For all the tasks, the models for spring layout graphs generally performed better than the models for random layout graphs. We show the test accuracy and loss in Tables 13 and 14 respectively.

Model	**Is Connected**		**Is Tree**		**Is Planar**	
	SL 50	RL 50	SL 50	RL 50	SL 50	RL 50
Simple NN (without pytorch)	0.9971	0.9150	0.9739	0.8675	0.9128	0.9327
BinaryModel	0.9996	0.9328	0.9975	0.8956	0.9511	0.9467
BinaryCEModel	0.9995	0.9343	0.9953	0.8962	0.9563	0.9486
CNN with Batch Normalization	0.9998	0.9382	0.9987	0.9303	0.9641	0.9523

Table 13: Test accuracy for 50 node across different graph types and layouts. Input images are 28×28. IC = Is Connected, IT = Is Tree, IP = Is Planar, SL = Spring Layout, RL = Random Layout.

Model	**Loss**	**Is Connected**		**Is Tree**		**Is Planar**	
		SL 50	RL 50	SL 50	RL 50	SL 50	RL 50
Simple NN (without pytorch)	MSE	0.0019	0.0606	0.0203	0.1007	0.0644	0.0503
BinaryModel	BCE with logits	0.0008	0.1558	0.0089	0.2691	0.1238	0.1344
BinaryCEModel	Cross Entropy	0.0013	0.1558	0.0141	0.2673	0.1066	0.1296
CNN with Batch Normalization	Cross Entropy	0.0002	0.1412	0.0044	0.1946	0.0897	0.1188

Table 14: Test loss values for 50 node graphs across different graph types and layouts. Input images are 28×28. SL = Spring Layout, RL = Random Layout.

For graphs containing 100 nodes, the easiest task for the models was the connectivity task with spring layout graphs while the hardest task was the tree recognition task with random layout graphs. For all the tasks, the models for spring layout graphs performed better than the models for random layout graphs. We show the test accuracy and loss in Tables 15 and 16 respectively.

Model	**Is Connected**		**Is Tree**		**Is Planar**	
	SL 100	RL 100	SL 100	RL 100	SL 100	RL 100
Simple NN (without pytorch)	0.9970	0.8802	0.9711	0.8550	0.9410	0.9229
BinaryModel	0.9994	0.9178	0.9984	0.9023	0.9756	0.9550
BinaryCEModel	0.9995	0.9235	0.9980	0.9030	0.9765	0.9552
CNN with Batch Normalization	0.9998	0.9446	0.9999	0.9233	0.9765	0.9615

Table 15: Test accuracy for 100 node graphs across different graph types and layouts. Input images are 56×56. IC = Is Connected, IT = Is Tree, IP = Is Planar, SL = Spring Layout, RL = Random Layout.

Model	Loss	Is Connected		Is Tree		Is Planar	
		SL 100	RL 100	SL 100	RL 100	SL 100	RL 100
Simple NN (without pytorch)	MSE	0.0025	0.0859	0.0217	0.1101	0.0455	0.0577
BinaryModel	BCE with logits	0.0015	0.2001	0.0052	0.2564	0.0675	0.1194
BinaryCEModel	Cross Entropy	0.0016	0.1815	0.0067	0.2527	0.0641	0.1174
CNN with Batch Normalization	Cross Entropy	0.0008	0.1295	0.0007	0.2063	0.0655	0.1102

Table 16: Test loss values for 100 node graphs across different graph types and layouts. Input images are 56×56. SL = Spring Layout, RL = Random Layout.

Most of the 50 node models performed better than the corresponding 20 node models even though they used the same image size of 28×28. The only task that performed significantly worse using 50 nodes was the connectivity random layout task. For the other tasks, the models performed similarly or better for 50 nodes. This might be due to the method that the graphs are generated.

For the counting task, we trained two CNN models using 128×128 images containing graphs with 10-100 nodes. For the first model, we generated graphs using spring layout and for second model, we generated graphs using random layout. We measured model performance using mean absolute error (MAE). For spring layout, the test MAE was 2.8205 meaning the model's prediction for the number of nodes in the graph was off by an average of 2.8205 nodes. For random layout, the test MAE was 6.6615.

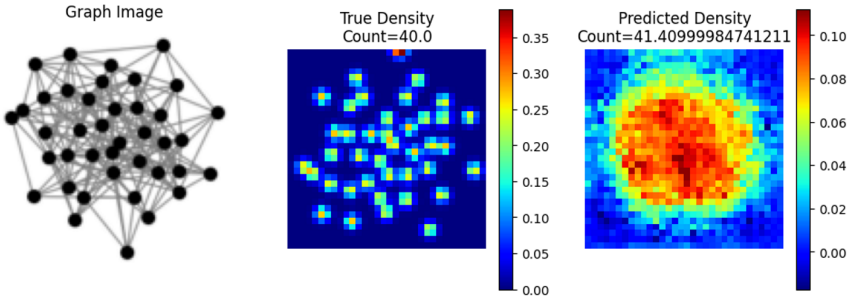

Figure 2: A sample image for the counting task using spring layout. The graph contains 40 nodes and the model was off by about 1.41 nodes.

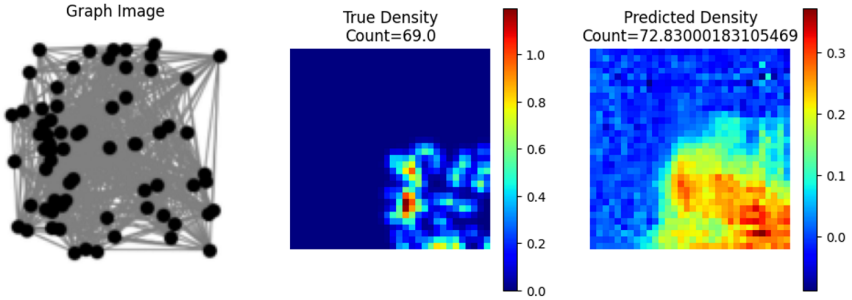

Figure 3: A sample image for the counting task using random layout. The graph contains 69 nodes and the model was off by about 3.83 nodes.

We use the integrated gradients technique to interpret the model outputs. The resulting images illustrate the reasoning behind the model's predictions (see Figures 4 and 5). In Figure 4, the left image is predicted as connected. The yellow regions highlight the edges responsible for connectivity, as well as other possible edges that would still maintain connectivity. The black regions in the corners indicate that the presence of isolated vertices in those positions would make the network disconnected. Similarly, the right image shows

vertex positions in the black regions that would cause the network to become disconnected. In Figure 5, the image shows that the yellow region in the top right corresponds to a connected component that is internally connected. The black dot in the bottom left indicates an isolated vertex that makes the network disconnected, while the other black dots represent additional vertices that would also keep the network disconnected. The yellow regions further indicate edges and vertices whose presence would make the network connected.

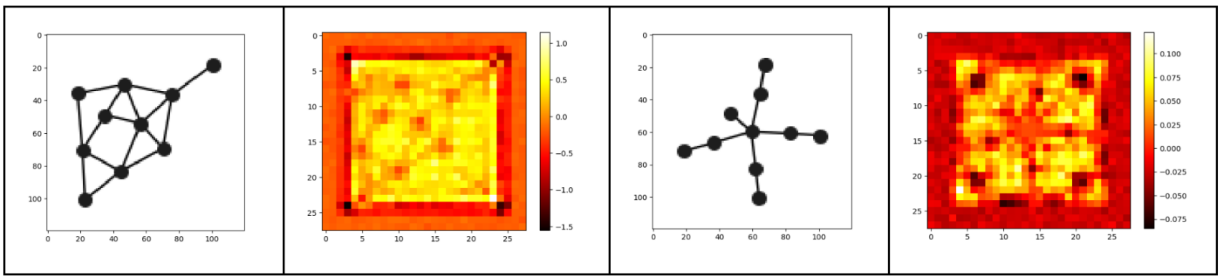

Figure 4: The integrated gradient images illustrate the reasoning behind the model's predictions. The left figure is predicted as connected. The yellow regions show the edges responsible for connectivity, as well as other possible edges that would still maintain connectivity. The black regions in the corners indicate that the presence of isolated vertices in those positions would make the network disconnected. Similarly, the right image shows possible vertex positions in the black regions that would cause the network to become disconnected.

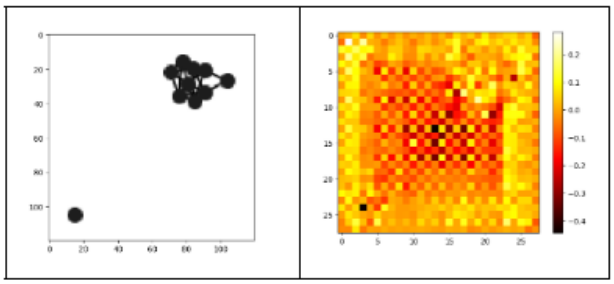

Figure 5: The integrated gradient image illustrates that the yellow region in the top right corresponds to a connected component that is internally connected. The black dot in the bottom left indicates an isolated vertex that makes the network disconnected. The other black dots indicate additional possible disconnected vertices that would also keep the network disconnected. The yellow regions further indicate possible edges and vertices, the presence of which would make the network connected.

3.5 Experimental Environment

All experiments were done with Google Colab which uses the Ubuntu 22.04.5 LTS operating system. Graph generation and simple NN training was done using Intel(R) Xeon(R) CPU (2.20 GHz) and 12.7 GB of RAM. All other models were trained using one T4 GPU.

4 Conclusion

In this work, we studied the ability of simple neural networks to infer structural properties of network layouts and to predict layout size from density distributions. Through extensive experiments on approximately one million generated samples, we demonstrated that even relatively simple architectures can learn meaningful representations of these properties. Our analysis of key design and training factors further highlighted the conditions that most strongly influence model performance. Building on these insights, we proposed a

unified framework capable of identifying multiple properties of network layouts across varying sizes. Finally, we employed feature attribution methods to interpret model predictions, providing additional insight into the learned decision-making process. Overall, our results suggest that neural networks can effectively capture structural information in network layouts, while interpretability techniques offer a valuable tool for understanding their behavior.

References

[1] Reyan Ahmed, Felice De Luca, Sabin Devkota, Stephen Kobourov, and Mingwei Li. Multicriteria scalable graph drawing via stochastic gradient descent, (sgd)2. *IEEE Transactions on Visualization and Computer Graphics*, 28(6):2388–2399, 2022.

[2] Reyan Ahmed, Felice De Luca, Sabin Devkota, Stephen Kobourov, and Mingwei Li. Graph drawing via gradient descent, $(gd)^2$. pages 3–17, 2020.

[3] Deepak Babu Sam, Shiv Surya, and R Venkatesh Babu. Switching convolutional neural network for crowd counting. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5744–5752, 2017.

[4] David Baehrens, Timon Schroeter, Stefan Harmeling, Motoaki Kawanabe, Katja Hansen, and Klaus-Robert Müller. How to explain individual classification decisions. *The Journal of Machine Learning Research*, 11:1803–1831, 2010.

[5] Sergey V Belim and Svetlana Yu Belim. Images segmentation based on cutting the graph into communities. *Algorithms*, 15(9):312, 2022.

[6] Mark Cheung and José MF Moura. Graph classification: Tradeoffs between deep neural network architecture and graph topology. In *2021 55th Asilomar Conference on Signals, Systems, and Computers*, pages 454–458. IEEE, 2021.

[7] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. *Introduction to algorithms*. MIT press, 2022.

[8] Sabin Devkota, Reyan Ahmed, Felice De Luca, Katherine E Isaacs, and Stephen Kobourov. Stress-plus-x (spx) graph layout. In *International Symposium on Graph Drawing and Network Visualization*, pages 291–304. Springer, 2019.

[9] Paul Erdős and Alfréd Rényi. On random graphs, i. *Publicationes Mathematicae (Debrecen)*, 6:290–297, 1959.

[10] Henry Forster, Felix Klesen, Tim Dwyer, Peter Eades, Seok-Hee Hong, Stephen Kobourov, Giuseppe Liotta, Kazuo Misue, Fabrizio Montecchiani, Alexander Pastukhov, et al. Graphtrials: Visual proofs of graph properties. *IEEE Transactions on Visualization and Computer Graphics*, 2025.

[11] Thomas MJ Fruchterman and Edward M Reingold. Graph drawing by force-directed placement. *Software: Practice and experience*, 21(11):1129–1164, 1991.

[12] Aric Hagberg, Pieter J Swart, and Daniel A Schult. Exploring network structure, dynamics, and function using networkx. Technical report, Los Alamos National Laboratory (LANL), 2007.

[13] Nora Hartsfield and Gerhard Ringel. *Pearls in graph theory: a comprehensive introduction*. Courier Corporation, 2013.

[14] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv:1412.6980*, 2014.

[15] Stephen G Kobourov. Spring embedders and force directed graph drawing algorithms. *arXiv preprint arXiv:1201.3011*, 2012.

[16] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.

[17] Yuhong Li, Xiaofan Zhang, and Deming Chen. Csrnet: Dilated convolutional neural networks for understanding the highly congested scenes. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1091–1100, 2018.

[18] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3431–3440, 2015.

[19] Youssef Mourchid, Mohammed El Hassouni, and Hocine Cherifi. A new image segmentation approach using community detection algorithms. In *2015 15th International Conference on Intelligent Systems Design and Applications (ISDA)*, pages 648–653. IEEE, 2015.

[20] Michael A. Nielsen. *Neural Networks and Deep Learning*. Determination Press, 2015.

[21] Albert Nijenhuis and Herbert S Wilf. *Combinatorial algorithms: for computers and calculators*. Elsevier, 2014.

[22] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[23] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115(3):211–252, 2015.

[24] Avanti Shrikumar, Peyton Greenside, Anna Shcherbina, and Anshul Kundaje. Not just a black box: Learning important features through propagating activation differences. *arXiv preprint arXiv:1605.01713*, 2016.

[25] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International Conference on Machine Learning (ICML)*, 2017.

[26] Xiaoqi Wang, Kevin Yen, Yifan Hu, and Han-Wei Shen. Smartgd: A gan-based graph drawing framework for diverse aesthetic goals. *IEEE Transactions on Visualization and Computer Graphics*, 30(8):5666–5678, 2023.

[27] Wenqian Ye, Guangtao Zheng, Xu Cao, Yunsheng Ma, and Aidong Zhang. Spurious correlations in machine learning: A survey. *arXiv e-prints*, pages arXiv–2402, 2024.

[28] Matthew D Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *European conference on computer vision*, pages 818–833. Springer, 2014.

[29] Yingying Zhang, Desen Zhou, Siqin Chen, Shenghua Gao, and Yi Ma. Single-image crowd counting via multi-column convolutional neural network. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 589–597, 2016.